

Types of communication

- Synchronous communication
 - Sender will *wait* until it is certain that the message is delivered
- Asynchronous communication
 - Sender continues immediately after submitting a message
- Persistent communication
 - Message that has been submitted for transmission is *stored* by the *communication middleware*
- Transient communication
 - Message is stored by the communication system only if the sending and receiving applications are executing

Client/Server

- Client/Server computing is generally based on a model of transient synchronous communication:
 - Client and server must be active at time of communication
 - Client issues request and blocks until it receives reply
 - Server waits only for incoming requests, and subsequently processes them
- Drawbacks of synchronous communication
 - Client cannot do any other work while waiting for reply
 - Failures must be handled *immediately*; the client is waiting
 - The model may simply not be appropriate (mail, news)

Messaging

- Message-oriented middleware
- Aims at high-level persistent asynchronous communication
- Processes send each other messages, which are *queued*
- Sender need not wait for immediate reply, but it can do other things
- Middleware often ensures *fault tolerance*

Procedure

- Call from one function to another function within the same process
 - Uses stack to pass arguments and return values
 - Accesses objects via pointers (e.g., C) or by reference (e.g., Java)
- Procedure call has exactly-once semantics
 - If process is alive, called function executed exactly once

Remote Procedure Call (RPC)

- Proposed by Birrell and Nelson in 1984
- RPC is intended to simplify the development of cross-address space and cross-machine interactions
- Interactions between two processes follow client-server model

Challenges

- Call from one function to another function, where caller and callee function reside in different processes
 - Function call crosses a process boundary
 - Accesses procedures via global references
 - Can't use pointers across processes since a reference address in process P1 may point to a different object in another process P2
 - E.g., Procedure address = IP + port + procedure number
- Parameters passed to the callee and only the results come back to the caller
- Different machines may represent numbers, characters, etc., in a different way:
 - Little endian e.g., x86, M1
- Advantages:
 - High-level interface for data movement and communication
 - Hiding complexities of cross-machine interactions from applications (*access transparency*)
 - Performs type checking, error handling and bytes interpretation, cross-machine conversion such big/small endian, etc.

RPC Call Semantics:

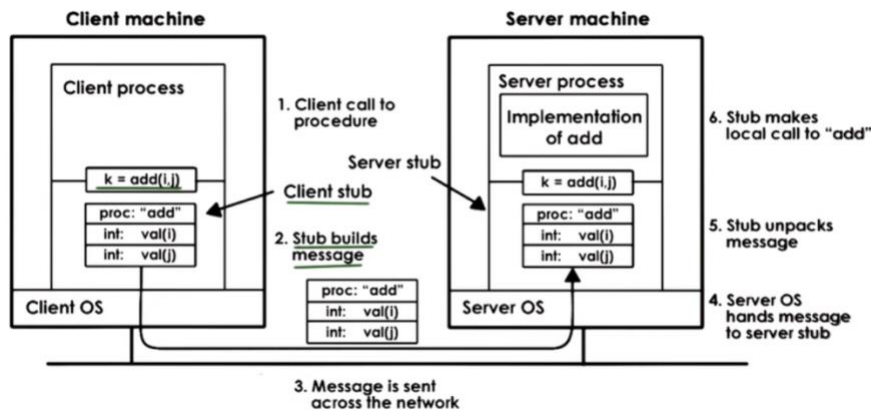
- Under failures, hard to guarantee exactly-once semantics
- Function may not be executed if
 - Request (call) message is dropped
 - Reply (return) message is dropped
 - Called process fails before executing called function
 - Called process fails after executing called function
- Hard for caller to distinguish these cases
- Function may be executed multiple times if request (call) message is duplicated

Implementing RPC Call Semantics:

- Possible semantics
 - At most once semantics (e.g., Java RMI)
 - At least once semantics (e.g., Sun RPC)
 - Idempotent operations can be used with at least once semantics
 - Maybe, i.e., best-effort (e.g., CORBA)

Retransmit request	Filter duplicate requests	Re-execute function or retransmit reply	RPC Semantics
Yes	No	Re-execute	At least once
Yes	Yes	Retransmit	At most once
No	NA	NA	Maybe

Structure of RPC



- | | |
|--|---|
| 1 Client procedure calls client stub. | 6 Server does local call; returns result to stub. |
| 2 Stub builds message; calls local OS. | 7 Stub builds message; calls OS. |
| 3 OS sends message to remote OS. | 8 OS sends message to client's OS. |
| 4 Remote OS gives message to stub. | 9 Client's OS gives message to stub. |
| 5 Stub unpacks parameters; calls server. | 10 Client stub unpacks result; returns to client. |

RPC steps

1. Register: server "registers" the supported procedure, argument types, its location, so that clients can discover it
2. Bind: client finds and "binds" to the desired server
 - o For a TCP-based protocol it involves the creation of a connection
3. Call: client makes an RPC call
 - o The control is passed to the client stub and client code blocks
4. Marshal: client stub "marshals" arguments (serialize arguments, which is distributed in the client's memory space) into a buffer
5. Send: client stub sends the message to the server
6. Receive: server receives the message, performs optional access control, and passes the message to the server stub
7. Unmarshal: server stub "unmarshals" arguments by extracting arguments from the message and create structs in the local memory
8. Actual call: server stub calls the local procedure implementation
9. Result: server performs the operation and computes the result of the RPC implementation
10. Return: it performs similar steps to return the result back to the client

Interface Definition Language (IDL)

- Client and server can be developed independently with different languages and requires the interface to be defined
- Helps serialize (converting objects to bytes) and deserialize (converting bytes to objects) to transfer data over the network
- More popular than
 - o Custom SER/DSER, XML, Java SER/DESER
- Describes the interface the server expects
 - o Procedure name, arguments, and result types
 - o Version number

RPC can use IDL that is

1. Language agnostic
 - o XDR in SunRPC
 2. Language specific
 - o Java in JavaRMI
- Note that the IDL language is only used by the RPC system to generate stubs, perform discovery, etc.
 - o It does not dictate the actual implementation

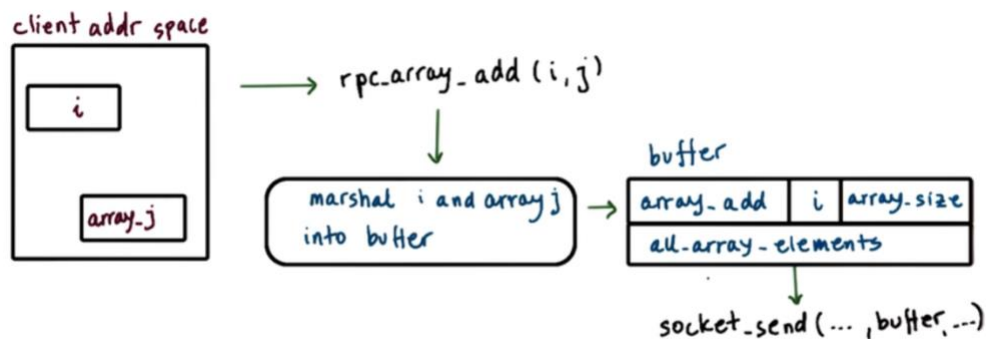
Registry and Binding

- A registry is a database of services available to a client
 - o It includes *service names* and *connection details* so that client can perform binding
 - o Registry can be distributed for any RPC service to register, or machine-specific so that only local clients or clients that know the machine address to query them
- The client requires information of available servers to perform binding
- During the binding step, client determines which server it should connect to based on the service name, version number, etc., and how to connect to the server, based on IP address, transport protocol, etc.

Code Generation

- Programmer only writes code for caller function and callee function
- Code for remaining components all generated automatically from function signatures (or object interfaces in Object-based languages)
 - o E.g., Sun RPC system: Sun XDR interface representation fed into rpcgen compiler
- These components are part of a Middleware system
 - o E.g., CORBA (Common Object Request Brokerage Architecture), Sun RPC, Java RMI

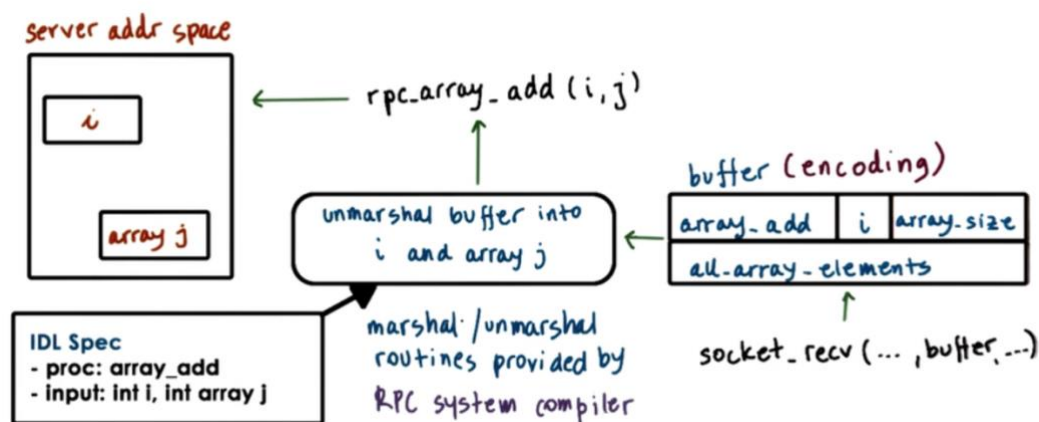
Marshaling



- Marshaling/Unmarshaling routines are provided by RPC system compiler
- When an RPC call is made on the client side, variables in the client application's memory space is passed to it as arguments, and they are not continuous in the memory

- However, at the lowest level, a `send()` call is issued to the socket to send the message across the network, and the `send()` call requires a buffer of continuous memory
- The marshalling code will create a buffer, copy arguments from various memory locations into the buffer, as well as other supporting information such as operation name, size of the argument, etc., using a pre-defined structure
- It also performs encoding if necessary
 - Different machines may represent numbers, characters, etc., in a different way:
 - little endian (least significant bit first) e.g., x86 chip,
 - big endian (highest significant bit first) e.g., M1 chip
 - Solution: Use common data representation

Unmarshaling



- Unmarshaling performs the opposite operation
- It decodes the message, extracts arguments from the message and allocate them into the local memory space, in preparation for invoking the implementation of the RPC call
- Note that marshal/unmarshal routes are provided by the RPC system compiler that generates the code from IDL to a specific language
- One tricky issue is the usage of pointers
 - RPC protocol can either prohibit the usage of pointers, or implement marshaling/unmarshaling for pointers so that the referenced data structure is included in the buffer

Basic RPC operation

Passing Value Parameters

- Client and server need to properly interpret messages, transforming them into machine-dependent representations
- Wrapping a parameter means transforming a value into a sequence of bytes
- Client and server must agree on the same encoding:
 - Representation of basic data types: integers, floats, characters
 - Representation of complex data types: arrays, unions

Pass Reference Parameters

One solution

- Forbid pointers and reference parameters

Other solution

- Call-by-copy/store
 - Copy the referenced data to the server and copy back the result from the server
 - Issue: Input data can be very large

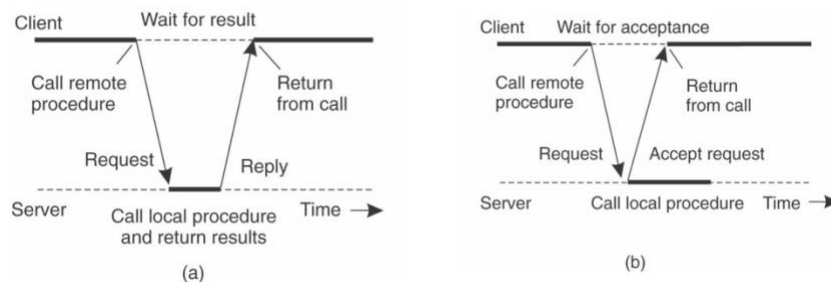
Types of RPCs

Synchronous RPC

- In synchronous RPC, the client makes the RPC call and waits for the result from the server

Asynchronous RPC

- In asynchronous RPC, the client makes the RPC call and waits for an acknowledgment from the server and continues
- On the other hand, after completing the computation the server sends the result of the call to the client



(a) Synchronous RPC

(b) Asynchronous RPC

Message-oriented Communication

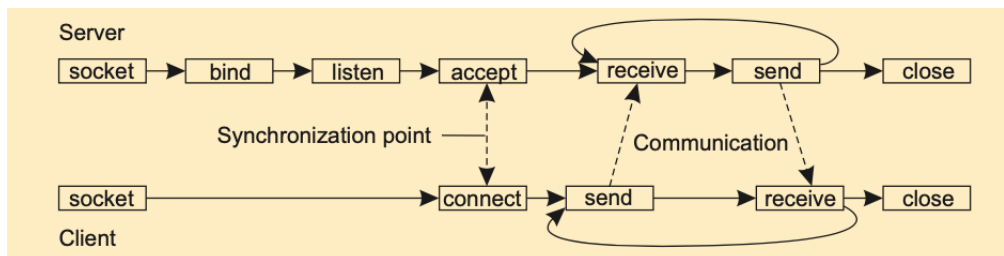
- RPC assumes that the server is active when a request is issued
- A client is often blocked until its request has been processed at the server
- Message-oriented communication is an alternative approach

Message-oriented Transient Communication

- Transport layer offers a simple message-oriented communication model
 - Many applications directly build on top of such a model
- A typical example is sockets
 - A socket is a communication endpoint for I/O at the transport layer
 - A server binds its IP address together with a port number to a socket

Berkeley socket interface

Operation	Description
socket	Create a new communication end point
bind	Attach a local address to a socket
listen	Tell operating system what the maximum number of pending connection requests should be
accept	Block caller until a connection request arrives
connect	Actively attempt to establish a connection
send	Send some data over the connection
receive	Receive some data over the connection
close	Release the connection



Message-passing Interface (MPI)

- Sockets only support simple communication primitives
- Sometimes we need a more powerful library for message passing
- Message-passing interface (MPI)
 - Mostly used for "transient" communication
 - MPI assumes communication takes place within a known group of processes

Some MPI Primitives

Operation	Description
MPI_bsend	Append outgoing message to a local send buffer
MPI_send	Send a message and wait until copied to local or remote buffer
MPI_ssend	Send a message and wait until transmission starts
MPI_sendrecv	Send a message and wait for reply
MPI_isend	Pass reference to outgoing message, and continue
MPI_issend	Pass reference to outgoing message, and wait until receipt starts
MPI_recv	Receive a message; block if there is none
MPI_irecv	Check if there is an incoming message, but do not block

Application Buffer

- Handled by the user

System Buffer

- Handled by the system
- Invisible to user

Blocking and Non-blocking Send

- A blocking send routine will only "return" after it is safe to modify the application buffer (your send data) for reuse
 - Safe does not imply that the data was actually received, it may still be in a system buffer
 - Can be synchronous which means there is handshaking occurring with the receive process to confirm a safe send
 - Can be asynchronous if a system buffer is used to hold the data for eventual delivery to the receive
- Non-blocking send and receive return almost immediately
 - They do not wait for any communication events to complete, such as message copying from user memory to system buffer space or the actual arrival of message
 - Non-blocking operations simply "request" the MPI library to perform the operation when it can
 - The user cannot predict when that will happen
 - It is unsafe to modify the application buffer (your variable space) until you know for sure the requested non-blocking operation was actually performed by the library
 - There are "wait" routines used to do this
 - Non-blocking communications are primarily used to overlap communication with computation to exploit possible performance gains

Message-oriented Persistent Communication

- MPI is transient

Message-queuing systems/message-oriented middleware (MOM)

- The essence of these systems is offer intermediate storage for messages
- Do not require sender/receiver to be active during transmission
- Takes longer time to transfer messages compared to sockets

Message-queuing Model

- Insert messages in specific queues
- Messages are sent through various communication servers (e.g., mail servers)
- No guarantee of when the message is delivered

Basic MQM primitives

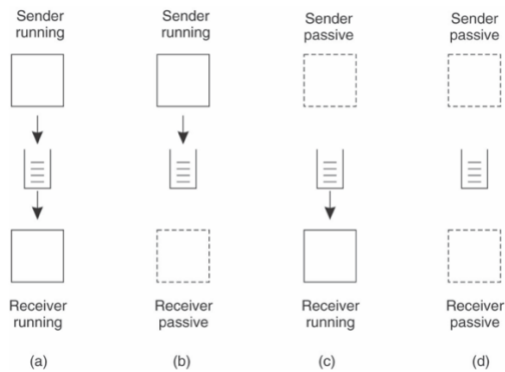
Put: append a message to a queue

Get: retrieve (remove) the first message of a queue

Poll: check a specific queue for messages

Notify: install a handler to be called when a message is put into the specified queue

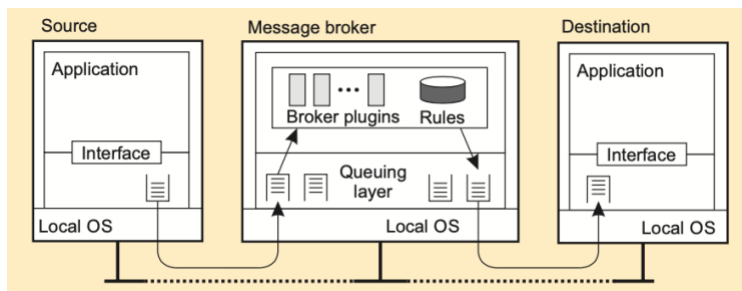
Four Combinations for MQM



Message Brokers

- Message-queuing systems can be used to integrate existing and new applications into a single, coherent distributed information systems
- Senders and receivers have to agree on the format of messages
- Message brokers
 - 3rd party company
 - The main task is to convert messages so that they can be understood by the receiving applications
 - Ex. IBM MQ

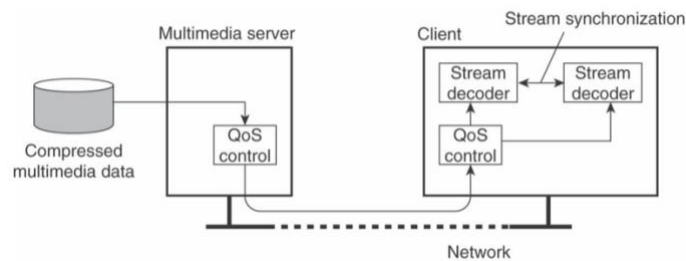
Message broker: general architecture



Stream-oriented Communication

- Communication discussed so far focuses on transmission of independent and complete unit of information
- The transmission of time-dependent information, e.g., video and audio streams are different
- Asynchronous transmission: streams transmitted one after the other (i.e., no constraint on when transmission should take place)
- Synchronous transmission: A maximum end-to-end delay is defined for each unit in a data stream
- Isochronous transmission: It is necessary that data items are transmitted on time e.g., bounded jitter

Streaming Stored Multimedia Data



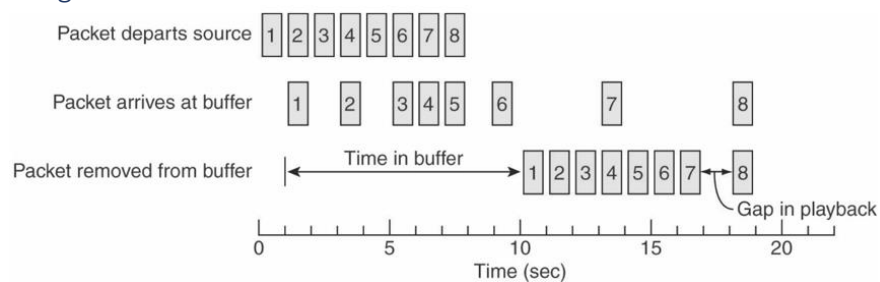
Important streaming QoS requirements

- The required bit rates
- The maximum delay until a session has been set up
- The maximum end-to-end delay
- The maximum delay variance, or jitter
- The maximum round-trip delay

Jitter:

- Jitter (variable delay) is used to describe the amount of inconsistency in latency across the network

Using a Buffer to Reduce Jitter



Interleaving Transmission

- Single packet containing multiple audio and video frames
- Each packet has four frames

